

Les types construits : les tuples

Dans notre premier cours sur Python, nous avons vu les différents types simples permettant de stocker de l'information : **int**, **float**, **bool**.

Le type **str** est aussi un type simple mais on peut sur ce type particulier accéder aux éléments de celui-ci. Par exemple si on crée une chaîne avec la commande `ch='bonjour'` alors l'expression `ch[0]` nous renverra la valeur `'b'` et l'expression `ch[3:7]` la valeur `'jour'` valeurs qui seront également du type `str`.

Avec ces types simples, nous pouvons définir des variables qui représenteront chacune **une** donnée avec **une** valeur. Ceci est suffisant pour traiter un petit nombre de données, mais lorsque nous allons vouloir traiter un plus gros volume de données, ceci peut devenir contraignant complexe à gérer.

Par exemple, si nous voulons gérer un ensemble de points en géométrie, chacun comporte une abscisse et une ordonnée, il nous faudra donc deux valeurs par objet, si nous gérons un train, il sera composé d'une locomotive et d'un ou plusieurs wagons (dont nous ne connaissons pas le nombre à l'avance).

Pour répondre à ce type de besoins, nous allons donc étudier trois nouveaux types d'objets : le type **tuple** (n-uplet), le type **list** (liste) et le type **dict** (dictionnaire). Ces types présentent des **points communs** mais aussi des **différences**, ils nous permettront d'adresser des besoins similaires et leur différences nous aideront à faire notre choix en fonction du problème à résoudre.

Définition

INFO

Un objet de type tuple est une suite ordonnée d'éléments qui peuvent être chacun de n'importe quel type. C'est une séquence d'éléments non modifiables.

Création

Pour créer un tuple, on utilise les parenthèses et on sépare les éléments par des virgules : `t = (3,7)` . Essayez sur la machine les commandes suivantes :

Python

```
t = (3,7)
t
type(t)

t=()
t

t=(4,)
t
type(t)
```

Bilan

SUCCESS

- Un tuple est une séquence (exemple de tuple => `tu = (1, 5, 3, 4)`)
- Chaque élément d'un tuple est identifié par un indice (1er élément indice 0, 2e élément indice 1,...)
- Pour accéder à un élément d'indice `i` on utilise la notation entre crochets : `tu[i]` (avec `tu` un tuple)
- Une fois créé, un tuple ne peut pas être modifié